Adaptive agent modes — the verification loop

One sentence. Quality does not come from telling an agent to be careful; it comes from a check suite the agent cannot edit, executed by the harness, whose failures are handed back until the work is genuinely green.

This is the measured architecture of release 0.4.0. Everything below was learned by spending real provider calls on a real proprietary task and letting the numbers overrule the design.

What the measurements say

The benchmark task is a shadow-replay of a real shipped fix: the NYRx Preferred Drug List parser rework from HermesAirflow history. The agent gets the pre-fix code and the same brief the engineer had; the grader compares against the fix that actually shipped, across eight behavioural dimensions. Model: `gpt-5.6-sol` at medium effort, identical across arms. Three trials per arm (a single trial is never a verdict — the same arm has swung 16 points between runs).

Arm	What it gets	Score	Tokens per trial
Unscaffolded control	The task, nothing else	77, 77, 77	272–316k
Loop only	The task + failing checks handed back, with guidance	89[1], **100, 100**	647k–1.21M
Lean scaffold + loop	Context pack + ledger + the same loop	**100, 100, 100**	1.54–2.68M

[1] That trial was invalidated by the protected-files tripwire — the agent edited a file it was not allowed to touch — and is reported for completeness.

100 is the ceiling: it means the agent reproduced the behaviour of the fix a human engineer shipped after several iterations. The loop reaches it. The bare agent never does, on any trial.

Three earlier campaigns (six fixtures, 17 calls) measured the opposite of what this environment used to believe: **prompt scaffolding bought nothing.** Stacking instructions, impact-map forms and adversarial-threat-model templates produced identical scores at 4–10.7× the token cost. Those forms are gone. What replaced them is below.

The three modes

Modes exist for **cost** and **governance**. They do not exist to make the model smarter — that job belongs to the loop.

Mode	When	What you get
lite	Advisory answers; trivial mechanical or doc edits (≤4 requirements, ≤2 files)	Minimal core, compact ledger, completion gate
standard	Everything else that mutates code — features, parsers, public contracts, security-sensitive changes, wide refactors, multi-session work	Precision context pack + **the verification loop**
critical	Only critical blast radius: deploys, credential rotation, production data rollouts, cross-system releases	standard + human scope gate + independent verifier

Only **consequence** selects a mode, and only critical consequence buys the human gate. Explaining, planning or dry-running a deploy is not a deploy. Two axes attach capabilities without changing the mode: **clarity** (an underspecified task gets spec synthesis with a frozen requirement ledger — a guardrail against silent scope-shrink, not a quality claim) and **continuity** (multi-session work gets durable checkpoints and handoff). **Breadth is telemetry only**: a 30-file rename is ordinary `standard` work, because the symbol sweep covers the consumer surface mechanically — width was measured and predicts nothing.

The verification loop

At prepare time — before a single token is spent — the harness snapshots the workspace and freezes a check suite. The agent implements, then runs:

```
python .agentic/verification_loop.py step --suite .agentic/check-suite.json --workspace .
```

Exit	Meaning	Next
0	Every independent check passes	Proceed to the completion gate
1	Failures remain	Read `.agentic/loop-feedback.json`, fix the causes, step again
2	Terminated — iteration budget or no progress	Stop. Escalate to a human with the remaining failures

The suite and its budgets are digest-frozen. Weakening a check, deleting one, or granting yourself more iterations fails the gate. **Adding** checks is always allowed. The completion gate re-runs the whole suite itself, so a green report the agent wrote proves nothing — only the re-execution counts.

The five kinds of check

- **acceptance** — a command that must exit 0 (public tests, frozen spec acceptance tests).
- **differential** — the same command runs against the pre-change snapshot and against your work; any output difference not declared as expected is a silent regression. This catches the class of bug that no amount of prompting caught: a field quietly changing while the task was about something else.
- **symbol-sweep** — every file referencing a symbol you touched must be changed or explicitly inspected with a recorded note. This is the deterministic answer to "the agent said it checked the consumers."
- **property** — invariants over real data samples. **Requirements that live in the data are still requirements**: the benchmark's hardest defect was a rule no task text mentioned and only the data revealed.
- **finding** — an independent verifier's finding, blocking until a proving command re-executes green or a human waives it in writing. Prose never closes a finding.

Guidance travels with failures, not with prompts

A failing check carries its own guidance — the relevant convention excerpt and the direction of the fix — into the feedback. A passing check carries nothing. This is just-in-time retrieval applied to verification: you pay for context exactly when something is red, and never otherwise.

It is also, empirically, the highest-leverage component in the system. Guidance attached to failures is what took the bare agent from 77 to 100.

Evidence is generated, not transcribed

Earlier versions made the agent write evidence rows, verification runs and a completion claim. Measurement killed that design: the heavyweight scaffold burned its turn budget on paperwork and ran out mid-fix, scoring 88 with a gate that honestly refused to certify it.

Since the harness re-executes every check anyway, **the harness writes the evidence**. The agent owns only what genuinely needs judgment: resolving material ambiguities, and recording what it actually verified at a consumer site. If a suite contains nothing executable, the gate falls back to demanding recorded verification commands — it never accepts nothing.

What it costs

Preparing the loop costs zero model tokens. The first iteration costs what an unscaffolded run costs. Further iterations happen **only when independent checks are red** — that is, exactly when the unscaffolded agent would have shipped the defect and you would have paid for it later, in production.

On the benchmark, reaching the ceiling cost 2–4× an unscaffolded run through the loop-only path and 5–9× through the lean scaffold path. On work the model already handles, the loop goes green on the first iteration and the overhead is a few seconds of CPU. Budget it as insurance that bills on claims, not as a subscription.

Benchmark protocol

Any claim about this environment must clear the same bar we hold ourselves to:

- **Three trials per arm minimum.** Single-trial results are exploratory and may never be reported as verdicts. The same arm scored 73 and 89 on identical inputs.
- **The canary rule.** A fixture with no valid paid history gets exactly one call on the cheapest arm; the main stage needs separate approval built from a validated canary record. The canary is not a trial.
- **The admission gate.** A fixture proves it discriminates before it costs money: the known-bad control must fail, the reference must pass, and graders must survive a hostile battery.
- **Checks are harness-side.** If the control arm can read the checks, you are measuring the checks as context, not measuring the loop. We learned this by contaminating a campaign and discarding 11 calls.
- **Every loop iteration is a counted provider call.**

Entry points

```
tools/route.ps1 -Adaptive -TaskFile <task> # mode decision + context pack

tools/adaptive/prepare_context.py # context, ledger, baseline snapshot, frozen check suite
tools/adaptive/verification_loop.py # the loop
tools/adaptive/harness_checks.py # the checks
tools/adaptive/pre_submit_gate.py # fail-closed completion gate (re-runs the suite)
```

Declare project-specific checks in `harness-checks.json` at your repo root: give each a `kind`, a `script`, and `guidance` that names the convention and the fix direction. Scripts are sha-pinned when the suite is frozen.

The classic router (`Router/catalog/modules.json`, default `tools/route.ps1`) is unchanged. The adaptive layer is additive and opt-in.

What we still do not know

- The loop is measured on one real task family. Replication on a second is the next honest step.
- The scaffold's contribution beyond the loop is not separated: `standard-loop` and `vanilla-loop` both reach 100, and three trials cannot resolve reliability differences that small.
- End-to-end browser checks and true adversarial multi-agent debate are described in the research corpus and are not built.
- Bare benchmark arms are not told which files are protected; three trials were invalidated on a rule the arm was never given.